RiskWatch

Real-Time Multi-Disease Risk Prediction System | Big Data Architecture Project

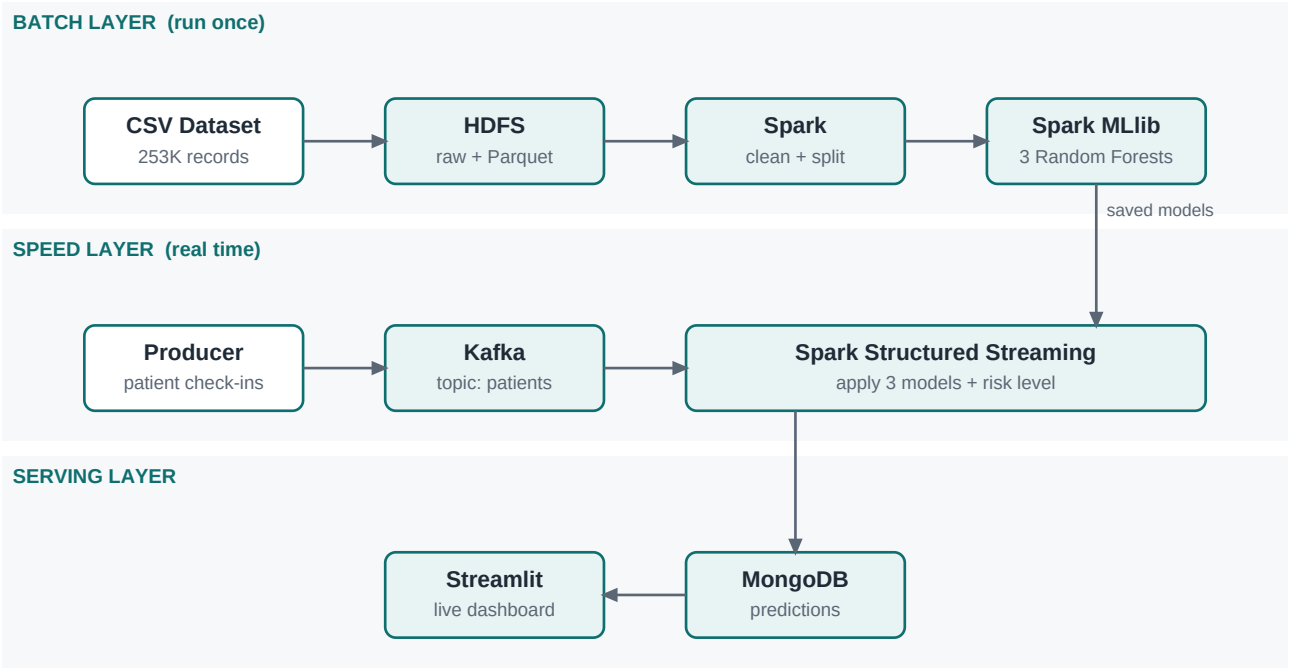
1. Problem and idea

Hospitals and screening camps see a steady flow of patients. Checking every person for several chronic diseases by hand is slow, so high-risk people can be missed. RiskWatch takes patient health records as they arrive, predicts the risk of **Diabetes, Heart Disease and Stroke** for each person within seconds, and flags high-risk patients on a live dashboard so staff know who to attend to first.

2. Objectives

- Store and process a large health dataset with distributed tools (HDFS, Spark).
- Train three disease prediction models with Spark MLlib.
- Ingest patient records in real time with Kafka and score them with Spark Structured Streaming.
- Store results in MongoDB and display live risk levels and alerts.

3. System architecture (Lambda architecture)



The **batch layer** learns from historical data. The **speed layer** applies what was learned to new data the moment it arrives. The **serving layer** stores and presents the results.

4. Dataset

CDC Diabetes Health Indicators (BRFSS 2015), free on Kaggle. It has about 253,680 records and 22 columns. Three columns are disease labels (*Diabetes_binary, HeartDiseaseorAttack, Stroke*) and the other 19 are shared inputs such as HighBP, HighChol, BMI, Smoker, PhysActivity, GenHlth, Age and Income. Because all three models use the same inputs, one incoming record can be scored for all three diseases at once.

5. Tech stack and the role of each tool

Layer	Tool	Why it is used
Storage	HDFS	Distributed, fault-tolerant storage for raw data, Parquet files and models

Batch processing	Apache Spark	Parallel, in-memory processing of the full dataset
Machine learning	Spark MLlib	Trains Random Forest models on distributed data
Ingestion	Apache Kafka	Buffers the live stream and decouples producers from processing
Stream processing	Spark Structured Streaming	Scores incoming records in micro-batches every 2 seconds
Database	MongoDB	Fast writes and flexible JSON documents for predictions
Visualization	Streamlit	Simple live dashboard written in Python
Deployment	Docker Compose	Starts all services with one command

6. Modules and data flow

#	Module (file)	What it does
1	Ingestion preprocess.py	Copies the CSV into the HDFS raw zone and reads it back with Spark.
2	Preprocessing preprocess.py	Casts columns, removes nulls and duplicates, shows disease prevalence, splits 80/20 and saves train/test as Parquet in HDFS.
3	Model training train_models.py	VectorAssembler builds the feature vector. One Random Forest (50 trees, depth 8) per disease, with class weights for imbalance. Evaluates AUC, accuracy, precision, recall and saves the models to HDFS.
4	Producer producer.py	Replays held-out patients as JSON messages to the Kafka topic *patients*, simulating hospital check-ins.
5	Stream prediction stream_predict.py	Reads Kafka, parses JSON with a schema, applies the 3 models, converts probability to Low / Medium / High, writes each micro-batch to MongoDB.
6	Dashboard app.py	Shows total patients, alerts, risk distribution, alerts over time, high-risk patient list and model performance. Refreshes every few seconds.

7. Risk levels

Predicted probability	Risk level	Action
below 30%	Low	Routine care
30% to 60%	Medium	Suggest follow-up tests
60% and above	High	Alert raised on dashboard

8. How it covers the 3 V's of Big Data

- **Volume:** about 253K records stored in HDFS and processed in parallel with Spark.
- **Velocity:** continuous stream of patients through Kafka, scored within seconds.
- **Variety:** CSV for raw data, Parquet for processed data, JSON events in Kafka, documents in MongoDB.

9. Evaluation metrics

- **AUC:** how well the model separates sick from healthy patients (0.5 = guessing, 1.0 = perfect).
- **Recall:** out of all truly sick patients, how many were caught. This matters most for screening.
- **Precision:** out of patients flagged, how many were truly sick. It is lower for rare diseases like stroke, which is acceptable for a screening tool because missing a patient is worse than an extra check.

Paste your own numbers from the training output (or the dashboard's Model performance section) into your report.

10. Likely viva questions

Question	Short answer
Why Lambda architecture?	Training needs all historical data (batch). Prediction must be instant (speed). Lambda keeps both separate and simple.
Why Kafka instead of sending data straight to Spark?	Kafka stores messages durably, so nothing is lost if Spark is slow or restarts. Producers and consumers are independent.
Why Spark and not pandas?	Spark spreads work across cores and machines and handles data larger than memory. The same code scales to a cluster.
Why Parquet?	Columnar and compressed, so Spark reads only needed columns and loads faster than CSV.
Why Random Forest?	Handles mixed feature types, resists overfitting, gives feature importance, and is built into MLlib.
How do you handle class imbalance?	Class weights: rare positive cases get a higher weight during training, which improves recall.
Why MongoDB?	Each prediction is a JSON-like document, writes are fast, and the schema can change without migrations.
What is a checkpoint?	Spark saves stream progress (Kafka offsets) so it can resume from where it stopped after a failure.
What is a micro-batch?	Spark groups records arriving in each 2-second window and processes them together.

11. Limitations and future scope

- Survey data, not clinical test results, so this is a screening tool and not a diagnosis tool.
- Future: real hospital or wearable data, more diseases (kidney, liver), SMS/email alerts, deployment on a cloud Spark cluster, model retraining on a schedule.

12. Conclusion

RiskWatch shows a complete Big Data pipeline: distributed storage, batch training, real-time ingestion, stream processing, NoSQL storage and live visualization. It turns a stream of patient records into instant, prioritised risk alerts for three chronic diseases using only free, open-source tools.